Objects in JS

Lec-7

Objects in JS

- Objects are **variable** that can store both **values** and **functions**.
- Values are stored as **key: value** pairs called as **properties**.
- Objects by default stores key and value in Strings.
- Functions are stored as **key:function()** pairs called **methods**.

Phone Properties	Phone Methods
phone.name = iPhone	phone.on()
phone.model = 17 Pro	phone.play()
phone.weight = 206 gm	phone.pause()
phone.color = Cosmic Orange	phone.off()

phone Object

- Different phones have the same **properties**, but the **property values** can differ from phone to phone.
- Different phones can have the same **methods**, but the methods can be performed **at different times**.
- **Eg** :

```
const phone = {
    name: "iPhone",
    model: "17 Pro",
    color: "Cosmic Orange"
};
```

- Here, **name**, **model** and **color** are **properties** and **"iPhone"**, **"17 Pro"**, **"Cosmic Orange"** are **property values**.
- Generally Objects can be created by the following 2 ways:

Way-1 : Using Object literal.

Way-2 : Using 'new' keyword

Using Object literal

```
const person = {
    firstName: "John",
    lastName:  "Doe",
    age: 18,
    sex: "male",
}
```

Using 'new' keyword

```
const person = new Object({
    firtName: "John",
    lastName: "Doe",
    age: 18,
    sex: "male"
});
```

Adding Properties in an object

- You can also create an **empty object**, and add properties later.
- **Eg**:

```
const person = {}
console.log(person);
person.firstName = "John"
person.lastName = "Doe"
person.age = 18
person.sex = "male"
```

Deleting Properties of an object

- The delete keyword deletes a property from an object
- **Eg**:

```
delete person.sex;
console.log(person);
```

Check if a Property Exists in an object

- Use the **in** property to check if a property exists in an object or not.
- **Eg**:

```
console.log("age" in person); // will print true
```

Accessing Object Properties

- You can access object properties in two ways:

1. **Dot notation**
2. **Bracket notation**

- **Eg**:

```
const car = {
   type: "Ferrari",
   model: "420 GTB",
   "car color": "Red",
   weight: "1,720kg"
}
```

```
console.log(car.type); // Ferrari
console.log(car.weight); // 1,720kg
```

Here accessing car color is not possible with the help of dot notation

```
console.log(car.car color); // This will result in SyntaxError
```

In such cases we tend to use bracket notation

```
console.log(car["car color"]); // Red
```

Preventing Changes in our Object Properties

- In order to prevent any changes to take place in our existing object, we use **Object.freeze()** method.
- **Eg** :

```
const movieObj = {
   "movie name": "Interstellar",
   "IMDB ratings": 8.7,
   "Released Year": 2014,
   "Streaming Platform": "Amazon Prime"
}
```

```
console.log(movieObj["Streaming Platform"]);
// will print Amazon Prime

movieObj["Streaming Platform"] = "Netflix"
console.log(movieObj["Streaming Platform"]);
// will print Netflix
```

Now let's freeze the object

```
Object.freeze(movieObj)

movieObj["Released Year"] = 2016
console.log(movieObj["Released Year"]);
// will result in typeError
```

> **Note** :
>
> - If an object is freezed and if we try to change any of it's properties, doing so will result in **typeError** only when **"use strict"** is used at the top of your script file.
> - Incase **"use strict"** is not used, it would **not result in typeError** instead it would **print the old value itself**.

- In order to know any object is in freezed state or not, we can use **Object.isFrozen()**, which returns boolean value i.e either **true** or **false**.
- **Eg** :

```
console.log(Object.isFrozen(movieObj));
// will return true
```

- Suppose you want to get all the **keys** of an object
- **Eg** :

```
console.log(Object.keys(movieObj));
// ['movie name', 'IMDB ratings', 'Released Year', 'Streaming Platform', 'watched']
```

- Suppose you want to get all the **values** of an object
- **Eg** :

```
console.log(Object.values(movieObj));
// ['Interstellar', 8.7, 2014, 'Netflix', ƒ]
```

- Suppose you want to get all the **entries** of an object.
- **Eg** :

```
console.log(Object.entries(movieObj));
// ['movie name', 'Interstellar']
// ['IMDB ratings', 8.7]
// ['Released Year', 2014]
// ['Streaming Platform', 'Netflix']
// ['watched', ƒ]
```

> **Note** : **Object.keys(), Object.values() and Object.entries() method return and store thier result in an Array.**

'this' keyword in JS

- The **this** keyword **refers to an object**.
- In Js, **this** is used to access the object that is calling a method.
- Whenever **this** is used inside an object method, **this** refers to the object.
- **Eg** :

```
const personDet = {
    firstName: "John",
    lastName: "Doe",
    age: 25,
    hellofunc: function(){
        console.log(`Weclome ${this.firstName} ${this.lastName}`);
    }
}
console.log(personDet.hellofunc());
```

- Here **firstName** and **lastName** cannot be accessed inside our **hellofunc()** without using this keyword since they're the properties of an object personDet and can only be accessed with the help of this keyword.
- **this.firstName** refers to the **firstName** property of the **personDet** object.
- **this.lastName** refers to the **lastName** property of the **personDet** object.
- Another reason for using **this** keyword, is we can use same method with different objects.
- **Eg** :

```
const person1 = {
    firstName: "Joe",
    lastName: "Biden",
    age: 69,
    hellofunc: function(){
        console.log(`Weclome ${this.firstName} ${this.lastName}`);
    }
}
console.log(person1.hellofunc());
// will print Welcome Joe Biden
```

```
onst person2 = {
    firstName: "Donald",
    lastName: "Duck",
    age: 65,
    hellofunc: function(){
        console.log(`Weclome ${this.firstName} ${this.lastName}`);
    }
}
console.log(person1.hellofunc());
// will print Welcome Donald Duck
```

- Whenever we use **this** keyword alone, it refers to the **global / supermost object**.
- In a browser, the supermost object is **window** object.
- In **node environment**, when used **this** keyword **alone** it refers to a **blank object {}**.
- If we use **arrow functions** inside an object, then it would give unexpected result compared to the above example.

```
const personDet = {
    firstName: "John",
    lastName: "Doe",
    age: 25,
    hellofunc: () =>{
        console.log(`Weclome ${this.firstName} ${this.lastName}`);
    }
}
console.log(personDet.hellofunc());
// will print "Welcome undefined undefined"
```

- In above example, **this** keyword does not refer to the personDet object.
- This is because arrow functions do not have thier own **this** value.
- Thus, **it's better to avoid using arrow functions inside an object**.

Practice Questions

Question-1 : E-commerce Inventory Tracker

You're adding a small inventory feature to a store project. Each product needs to live as an object so the catalog page can display its details and the warehouse team can update stock without touching the code directly.

- Create a product object literal named product with properties: **name, brand, price, stock count**, and **category**.
- Log the product's **name** to the console.
- A new shipment just arrived — add a new property called **restockDate** to the object.
- The product just got discontinued, so delete the **category** property from the object. Before deleting it, check whether a **warranty** property exists on the object and log the result.
- Finally, log the product's full structure **(keys, values and entries)**

Question 2: Player Bio Cards for Your World Cup Predictor

You're extending your World Cup Predictor project to show a short bio whenever a player card is clicked. Instead of writing a separate console log for every player, you want one reusable method that adapts based on whichever object calls it.

- Create two player objects, e.g. **player1** and **player2**, each with **name, club, position**, and **goals** properties.
- Give **each object** a **method** called **bio** that uses this to **log** something like **"Vinícius Júnior plays as a Forward for Real Madrid and has scored 24 goals this season."**
- Call bio() on both objects and confirm the same method logic produces different output depending on which object called it.

Question 3: Locking In a Flight Booking Price

You're building a flight booking feature where, once a customer confirms their seat, the displayed price should never silently change due to a bug somewhere else in the app, even if some other function tries to touch it.

- Create a booking object with **passengerName**, **flightNumber**, **seat**, and **price** properties.
- Log the **price** to the console
- Inside a variable **isBookingConfirmed** store the result on basis of booking is confirmed or not
- Freeze the object with once the booking is confirmed, and verify it's frozen or not.
- Add **"use strict"** at the top of your file, try changing the price one more time, and describe what error shows up and why the behavior changed.